



Cyberscope

A *TAC Security* Company

Audit Report

Fortune Protocol

April 2026

SHA256 :

05e42c27da72fe821e35d591a5fc260d1abd1d4a1a1423d5fc51e85be8f2ef74

Audited by © cyberscope

Table of Contents

Table of Contents	1
Risk Classification	3
Review	4
Audit Updates	4
Source Files	4
Overview	5
FRX (TestToken)	5
Analytics	6
Diagnostics	7
Findings Breakdown	8
ST - Stops Transactions	9
Description	9
Recommendation	11
MT - Mint Tokens	12
Description	12
Recommendation	13
CCR - Contract Centralization Risk	14
Description	14
Recommendation	14
DDP - Decimal Division Precision	16
Description	16
Recommendation	16
FRMV - Function Returns Misleading Values	17
Description	17
Recommendation	18
MSO - Manipulable Spot-Reserve Oracle	19
Description	19
Recommendation	21
MVN - Misleading Variables Naming	22
Description	22
Recommendation	22
MC - Missing Check	23
Description	23
Recommendation	24
MEE - Missing Events Emission	25
Description	25
Recommendation	25
PBOI - Phase Boundary Operator Inconsistency	27
Description	27

Recommendation	29
PNR - Privileges Not Revoked	30
Description	30
Recommendation	30
SLB - Sell Limit Bypass	31
Description	31
Recommendation	31
SPO - Spot Price Oracle Manipulation	32
Description	32
Recommendation	32
UM - Unusable Modifier	33
Description	33
Recommendation	34
L04 - Conformance to Solidity Naming Conventions	35
Description	35
Recommendation	36
Functions Analysis	37
Inheritance Graph	38
Flow Graph	39
Summary	40
Disclaimer	41
About Cyberscope	42

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Audit Updates

Initial Audit	09 Apr 2026 https://github.com/cyberscope-io/audits/blob/main/fortune-protocol/v1/audit.pdf
Corrected Phase 2	17 Apr 2026

Source Files

Filename	SHA256
FRX-1.sol	05e42c27da72fe821e35d591a5fc260d1abd1d4a1a1423d5fc51e85be8f2ef74

Overview

FRX (TestToken)

TestToken is a non-upgradeable ERC20 token with built-in transfer taxes and trading restrictions. Buy and sell transactions are subject to a 5% tax (1% burn, 2% to recovery contract, 2% to fee receiver), while regular transfers incur a 1% burn. Trading is time-gated: buys are disabled for the first 40 days after deployment and sells for the first 45 days. During the first 90 days, a cumulative \$500 buy limit per address is enforced. A permanent \$500 daily sell limit per address applies to all non-excluded users. The `pancakeRouter` address controls the pair setting and exclusion mapping, which bypasses all taxes, delays, and limits. The `pancakeRouter` can mint up to 80% of the maximum supply with a limit of 5% every month. The token uses Ownable for ownership management.

Analytics

● Critical ● Medium ● Minor / Informative

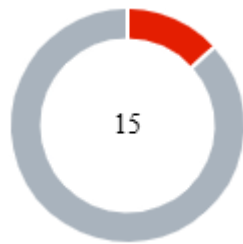
Severity	Code	Description	Status
●	ST	Stop Transactions	Unresolved
●	OTUT	Transfers User's Tokens	Passed
●	ELFM	Exceeds Fees Limit	Passed
●	MT	Mint Tokens	Unresolved
●	BT	Burns Tokens	Passed
●	BC	Blacklists Addresses	Passed

Diagnostics

● Critical ● Medium ● Minor / Informative

Severity	Code	Description	Status
●	CCR	Contract Centralization Risk	Unresolved
●	DDP	Decimal Division Precision	Unresolved
●	FRMV	Function Returns Misleading Values	Unresolved
●	MSO	Manipulable Spot-Reserve Oracle	Unresolved
●	MVN	Misleading Variables Naming	Unresolved
●	MC	Missing Check	Unresolved
●	MEE	Missing Events Emission	Unresolved
●	PBOI	Phase Boundary Operator Inconsistency	Unresolved
●	PNR	Privileges Not Revoked	Unresolved
●	SLB	Sell Limit Bypass	Unresolved
●	SPO	Spot Price Oracle Manipulation	Unresolved
●	UM	Unusable Modifier	Unresolved
●	L04	Conformance to Solidity Naming Conventions	Unresolved

Findings Breakdown



● Critical	2
● Medium	0
● Minor / Informative	13

Severity	Unresolved	Acknowledged	Resolved	Other
● Critical	2	0	0	0
● Medium	0	0	0	0
● Minor / Informative	13	0	0	0

ST - Stops Transactions

Criticality	Critical
Location	FRX-1.sol#L1446
Status	Unresolved

Description

The transactions are initially disabled for all users excluding the authorized addresses. The buy and sell transactions are disabled for the first 40 days (`TRADE_DELAY`). Once the transactions are enabled, the delays cannot be changed. Furthermore, the `pancakeRouter` has the authority to stop transactions for all users excluding the excluded addresses. The `pancakeRouter` may take advantage of it by setting the pair address via `setPancakeV2Pair` to a contract that does not implement the expected interface. As a result, calls to `_usdValue` will revert on `getReserves` , blocking all sell transactions. Once the contract is set to the correct address it cannot be changed again. Additionally, the `setExcluded` function allows the `pancakeRouter` to grant any address exclusion status, which bypasses all protections including taxes, delays, limits, and minimum holding requirements. The sell transactions are further limited by the `_enforceSellLimit` function, which restricts each non-excluded address to \$10 USD per day during days 41 through 90, \$100 USD per day during days 91 through 180. As a result, the contract may operate as a honeypot.

Shell

```
function _usdValue(uint256 tokenAmount) internal
view returns (uint256) {
    address _pair = pancakeV2Pair;
    if (_pair == address(0) || tokenAmount == 0)
return 0;

    IPancakeV2Pair pair = IPancakeV2Pair(_pair);
    (uint112 reserve0, uint112 reserve1, ) =
pair.getReserves();
    bool tokenIsToken0 = (pair.token0() ==
address(this));
    address quoteToken = tokenIsToken0 ?
pair.token1() : pair.token0();
    uint8 qDec =
IERC20Decimals(quoteToken).decimals();

    uint256 tokenReserve = tokenIsToken0 ?
uint256(reserve0) : uint256(reserve1);
    uint256 quoteReserve = tokenIsToken0 ?
uint256(reserve1) : uint256(reserve0);

    if (tokenReserve == 0) return 0;

    uint256 rawQuoteAmount = (1e18 * quoteReserve)
/ tokenReserve;

    uint fixedDecimals = qDec <= 18
? rawQuoteAmount * (10 ** (18 - qDec))
: rawQuoteAmount / (10 ** (qDec - 18));

    return (fixedDecimals * tokenAmount)/1e18;
}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions. Some suggestions are:

- Introduce a multi-sign wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

MT - Mint Tokens

Criticality	Critical
Location	FRX.sol#L1341
Status	Unresolved

Description

The contract grants the `pancakeRouter` address the authority to mint tokens. The `pancakeRouter` may take advantage of it by calling the `mint` function and mint 5% of the supply every month. As a result, the contract tokens will be highly inflated, up to 80% of `MAX_SUPPLY` in a duration of 16 months.

Shell

```
function mint(address to, uint256 amount) external {
    bool isOwner = (msg.sender == pancakeRouter);

    require(isOwner, "Not authorized to mint");

    require(
        block.timestamp >= deploymentTime +
        MINT_UNLOCK,
        "Owner minting locked"
    );

    require(monthlyMinted[getCurMonth()] + amount <=
        (MAX_SUPPLY * MONTH_MAX_MINT)/BPS, "max mint
        reached");
    require(totalSupply() + amount <= MAX_SUPPLY,
        "Exceeds max supply");

    _mint(to, amount);
    monthlyMinted[getCurMonth()] += amount;
}
```

Recommendation

The team should carefully manage the private keys of the pancakeRouter account. A timelock mechanism should be applied to the recovery contract's minting path, and per-period mint amounts should be capped to limit the potential for supply inflation.

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	FRX.sol#L1271,L1317,L1322,L1327
Status	Unresolved

Description

The contract's functionality and behavior are heavily dependent on a single externally controlled address. The pancakeRouter/owner address controls the pair setting via `setPancakeV2Pair` and the exclusion mapping via `setExcluded`, which bypasses all protections including taxes, delays, limits, and minimum holding requirements. The `recoveryContract` address can mint up to 80% of the maximum supply without the timelock that restricts the owner. None of these addresses have setter functions, meaning a key compromise is unrecoverable.

Centralization risks arising from the dependence on these addresses include single point of control, vulnerability to key compromise, and decentralization erosion.

Shell

```
function setPancakeV2Pair(address _pair) external
function setExcluded(address account, bool excluded,
uint _type) external

function mint(address to, uint256 amount) external
```

Recommendation

The team should carefully manage the private keys of the administrative account. To mitigate centralization risks, it is recommended to introduce a timelock mechanism with a reasonable delay, a multi-signature wallet so that multiple addresses must confirm actions, and bounds validation on all configurable parameters. Renaming the administrative variable

to reflect its actual role would improve transparency. Renouncing the role would eliminate the threats but is non-reversible.

DDP - Decimal Division Precision

Criticality	Minor / Informative
Location	FRX.sol#L1399,L1412
Status	Unresolved

Description

Division of decimal (fixed point) numbers can result in rounding errors due to the way that division is implemented in Solidity. For very small token amounts, `_usdValue` rounds to 0, trivially passing the \$500 daily sell limit. Similarly, the tax computation rounds to 0 for amounts below 50 wei. These rounding errors are economically negligible and do not pose a practical risk.

```
Shell
uint256 rawQuoteAmount = (tokenAmount * quoteReserve)
/ tokenReserve;

uint256 burnAmt      = (amount * TAX_BURN)      / BPS;
```

Recommendation

The team is advised to take into consideration the rounding results that are produced from the solidity calculations. The contract could calculate the subtraction of the divided funds in the last calculation in order to avoid the division rounding issue.

FRMV - Function Returns Misleading Values

Criticality	Minor / Informative
Location	FRX-1.sol#L1492
Status	Unresolved

Description

`remainingBuyAllowanceUSD()` is intended as a reliable indicator of how much a given account can still buy, but it produces incorrect values in two distinct scenarios:

Pre-trading window (`elapsed < TRADE_DELAY`): The function returns `type(uint256).max`, implying unlimited buy capacity. In reality, `_update()` reverts all buys before `TRADE_DELAY` elapses. Any caller treating the return value as "can this account buy?" will receive a false positive during this window. Excluded accounts (other than `recoveryContract`): The function only special-cases `recoveryContract` to return `type(uint256).max`. However, accounts in the `isExcluded` mapping bypass the buy-limit path in `_update()` entirely, so the limit computed by `remainingBuyAllowanceUSD()` does not reflect their actual on-chain limit. This is inconsistent with `remainingSellAllowanceUSD()`, which works correctly for excluded accounts with `if (isExcluded[account]) return type(uint256).max`.

Shell

```
function remainingBuyAllowanceUSD(address account)
external view returns (uint256) {
    if (account == recoveryContract) return
    type(uint256).max;
    uint256 elapsed = block.timestamp -
    deploymentTime;
    if (elapsed < TRADE_DELAY || elapsed >=
    LIMIT_WINDOW) return type(uint256).max;
    uint256 used = cumulativeBoughtUSD[account];
    return used >= MAX_BUY_USD ? 0 : MAX_BUY_USD -
    used;
```

```
}
```

Recommendation

Add a trading-active guard and a general exclusion check at the top of `remainingBuyAllowanceUSD()` to make its semantics consistent with actual enforcement.

MSO - Manipulable Spot-Reserve Oracle

Criticality	Minor / Informative
Location	FRX-1.sol#L1452
Status	Unresolved

Description

The `_usdValue` function computes the USD value of a token amount by reading current reserves from the PancakeSwap V2 pair via `getReserves()` and using them as an instantaneous oracle. This value drives the `MAX_BUY_USD` cap enforced in the buy branch of `_update` and the daily `DAILY_SELL_USD_90_DAYS` and `DAILY_SELL_USD_180_DAYS` caps enforced inside `_enforceSellLimit`, so whoever influences pair state within a block controls whether a given swap passes or fails the limit checks. Bypassing the caps requires pushing the oracle to report a lower USD-per-token, which in turn requires increasing `tokenReserve` or decreasing `quoteReserve`.

- Any address in `isExcluded` — the liquidity wallet, recovery contract, fee receiver, router, or any address added by the privileged `setExcluded` caller — bypasses `_enforceSellLimit` entirely and can dump tokens into the pair without tripping its own cap, depressing the oracle reading for a non-excluded accomplice wallet's buy or sell in the same block.
- `getReserves()` returns the reserves committed at the end of the last swap, so a searcher can order a buy or sell immediately after any unrelated large sell and read a depressed state against which the cap check passes.

Shell

```
function _usdValue(uint256 tokenAmount) internal view
returns (uint256) {
    address _pair = pancakeV2Pair;
    if (_pair == address(0) || tokenAmount == 0)
return 0;

    IPancakeV2Pair pair = IPancakeV2Pair(_pair);
    (uint112 reserve0, uint112 reserve1, ) =
pair.getReserves();

    bool tokenIsToken0 = (pair.token0() ==
address(this));
    address quoteToken = tokenIsToken0 ?
pair.token1() : pair.token0();
    uint8 qDec =
IERC20Decimals(quoteToken).decimals();

    uint256 tokenReserve = tokenIsToken0 ?
uint256(reserve0) : uint256(reserve1);
    uint256 quoteReserve = tokenIsToken0 ?
uint256(reserve1) : uint256(reserve0);

    if (tokenReserve == 0) return 0;

    uint256 rawQuoteAmount = (1e18 * quoteReserve) /
tokenReserve;

    uint fixedDecimals = qDec <= 18
? rawQuoteAmount * (10 ** (18 - qDec))
: rawQuoteAmount / (10 ** (qDec - 18));

    return (fixedDecimals * tokenAmount)/1e18;
}
```

Recommendation

The team is advised to replace the spot-reserve price read with a manipulation-resistant source. This will decouple buy and sell cap enforcement from instantaneous pair state, making single-block dumps and MEV-ordered sandwiches economically unattractive and removing the leverage that excluded addresses currently have to crash the oracle on behalf of accomplice wallets.

MVN - Misleading Variables Naming

Criticality	Minor / Informative
Location	FRX.sol#L1271
Status	Unresolved

Description

The contract declares a variable that uses a name that misrepresents its actual semantics. The `pancakeRouter` variable suggests the PancakeSwap DEX router. In reality it is an arbitrary admin address set at construction that controls the pair setting and exclusion mapping (see CCR). The name obscures the centralization risk from users and auditors reviewing access control.

Shell

```
address private pancakeRouter;
```

Recommendation

The team is advised to replace misleading names with descriptive alternatives that accurately reflect each variable's or function's actual purpose. This improves code readability, reduces the likelihood of errors during future development, and provides greater transparency to users and auditors reviewing the contract.

MC - Missing Check

Criticality	Minor / Informative
Location	FRX.sol#L1274,1289,1325
Status	Unresolved

Description

The cumulativeBoughtUSD mapping is a flat mapping(address => uint256) with no time windowing or reset mechanism, unlike the daily sell limit which uses getCurDay buckets. Once a user buys \$500 during the 90-day window, they can never re-buy even after selling, creating a one-directional trap.

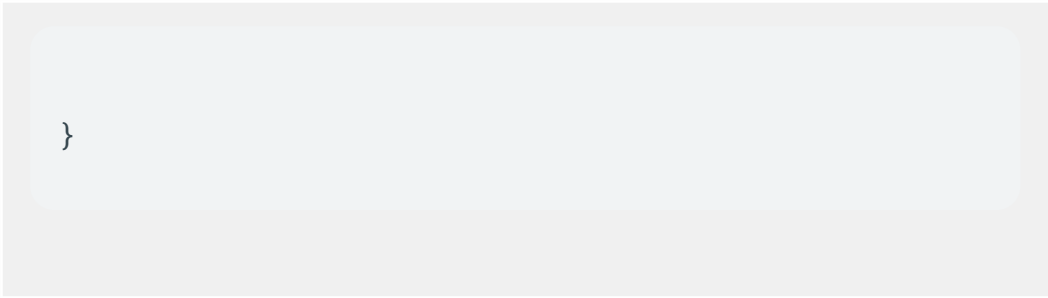
Shell

```
mapping(address => uint256) public
cumulativeBoughtUSD;
```

Additionally the `setPancakeV2Pair` does not ensure that the `_pair` is actually the correct pair needed for `_usdValue` to work properly

Shell

```
function setPancakeV2Pair(address _pair) external {
    require(_pair != address(0), "Zero: pair");
    require(msg.sender == owner(), "Not authorized");
    require(pancakeV2Pair == address(0), "Pair
already Set");
    pancakeV2Pair = _pair;
```



Recommendation

The team is advised to properly check the variables according to the required specifications. A periodic reset mechanism should be added to the cumulative buy limit consistent with the sell limit's daily bucketing approach.

MEE - Missing Events Emission

Criticality	Minor / Informative
Location	FRX.sol#L1317
Status	Unresolved

Description

The contract performs actions and state mutations from external methods that do not result in the emission of events. Neither `setPancakeV2Pair` nor `setExcluded` emits events.

Changes to the pair address and the exclusion mapping are invisible to off-chain monitoring. Without these events, it may be difficult for external parties to accurately determine the current state of the contract or detect suspicious administrative activity.

Shell

```
function setPancakeV2Pair(address _pair) external {
    require(_pair != address(0) && msg.sender ==
pancakeRouter, "Zero: pair");
    pancakeV2Pair = _pair;
}
function setExcluded(address account, bool excluded)
external {
    require(account != address(0) && msg.sender ==
pancakeRouter, "Zero: account");
    isExcluded[account] = excluded;
}
```

Recommendation

It is recommended to include events that are triggered each time a significant action takes place within the contract. These events should include relevant details such as the caller's

address and the parameters changed. This will make the contract more transparent and easily auditable by external parties.

PBOI - Phase Boundary Operator Inconsistency

Criticality	Minor / Informative
Location	FRX-1.sol#L1475,1508
Status	Unresolved

Description

`tradingInfo()` uses strict inequalities (`<`) to determine active sell-limit phases, while both `_enforceSellLimit()` and `remainingSellAllowanceUSD()` use inclusive inequalities (`<=`). This creates a one-block divergence at two critical timestamps:

At exactly `deploymentTime + 180 days` : `tradingInfo()` reports `sellUnlimited = true` and returns `type(uint256).max` , while enforcement still caps sells at `DAILY_SELL_USD_180_DAYS` . At exactly `deploymentTime + 90 days` : `tradingInfo()` reports Phase 2 as active and advertises the more lenient `DAILY_SELL_USD_180_DAYS` limit, while enforcement still applies the stricter `DAILY_SELL_USD_90_DAYS` cap.

In both cases, any UI, off-chain monitor, or integrating contract relying on `tradingInfo()` as a pre-flight check will present users with a more permissive sell allowance than what the contract actually enforces, potentially causing unexpected reverts.

Shell

```
function remainingSellAllowanceUSD(address account)
external view returns (uint256) {
    if (isExcluded[account]) return
type(uint256).max;

    uint256 elapsed = block.timestamp -
deploymentTime;
    if (elapsed < SELL_DELAY) return 0;

    uint sellLimit = type(uint256).max;
    if(block.timestamp - deploymentTime <=
LIMIT_WINDOW) {
        sellLimit = DAILY_SELL_USD_90_DAYS;
    } else if(block.timestamp - deploymentTime <=
SELL_LIMIT_WINDOW) {
        sellLimit = DAILY_SELL_USD_180_DAYS;
    }

    uint256 used =
cumulativeSoldUSD[account][getCurDay()];
    return used >= sellLimit ? 0 : sellLimit - used;
}

function tradingInfo() external view returns (
    ...
    sellLimitPhase1Active = elapsed >= SELL_DELAY &&
elapsed < LIMIT_WINDOW;
    sellLimitPhase2Active = elapsed >= LIMIT_WINDOW
&& elapsed < SELL_LIMIT_WINDOW;
    sellUnlimited = elapsed >= SELL_LIMIT_WINDOW;
    ...
}
```

Recommendation

Align `tradingInfo()` boundary conditions with the `<=` semantics used in enforcement.

Apply the same correction to the `currentDailySellLimitUSD` branching block.

PNR - Privileges Not Revoked

Criticality	Minor / Informative
Location	FRX-1.sol#L1137
Status	Unresolved

Description

This finding is used when the contract grants a privilege to an address (ig. exclude from fees) and when the address is changed to a new one, the privileges are not revoked from the previous address. Additionally, the new `recoveryContract` is not excluded from fees.

```
Shell
} else if(_type == 1) {
    recoveryContract = account;
}
```

Recommendation

The contract grants specific addresses certain privileges, providing operational flexibility. However, the current implementation does not automatically remove these privileges when an address's status changes. Consequently, the privileges remain active for addresses that no longer hold the specified roles or ownership. This oversight can lead to unintended discrepancies in privileges and poses potential security risks.

SLB - Sell Limit Bypass

Criticality	Minor / Informative
Location	FRX.sol#L1399
Status	Unresolved

Description

The \$500/day sell limit is tracked per-address via `cumulativeSoldUSD[seller][bucket]`. A holder can distribute tokens across multiple wallets (incurring a 1% transfer burn per hop) and sell from each. Splitting \$50,000 across 100 addresses costs approximately \$500 in transfer burns, then each wallet sells \$500/day for a total of \$50,000 in one day. The per-address tracking provides no protection against this sybil-based bypass.

```
Shell
function _enforceSellLimit(address seller, uint256
tokenAmount) private {
    uint256 usdVal = _usdValue(tokenAmount);
    uint256 bucket = getCurDay();
    uint256 newCum =
cumulativeSoldUSD[seller][bucket] + usdVal;
    require(newCum <= MAX_SELL_USD, "Exceeds $500
sell limit");
    cumulativeSoldUSD[seller][bucket] = newCum;
}
```

Recommendation

The team is advised to document this as a known limitation or to add a minimum holding period after receiving transfers. This would make address-splitting attacks less practical by requiring each new address to hold tokens for a period before selling.

SPO - Spot Price Oracle Manipulation

Criticality	Minor / Informative
Location	FRX.sol#L1422
Status	Unresolved

Description

The `_usdValue` function uses instantaneous spot reserves from `getReserves` to determine the USD value of a token amount. These spot reserves are manipulable via flash loans within a single transaction. An attacker can flash-borrow to skew the reserves, deflating the reported USD value, and then execute a buy or sell that bypasses the limit.

```
Shell
function _usdValue(uint256 tokenAmount) internal view
returns (uint256) {
    (uint112 reserve0, uint112 reserve1, ) =
    pair.getReserves();
    ...
    uint256 rawQuoteAmount = (tokenAmount *
    quoteReserve) / tokenReserve;
    ...
}
```

Recommendation

The team is advised to replace the spot price oracle with a time-weighted average price (TWAP) oracle or an external oracle such as Chainlink. This would prevent single-transaction price manipulation from bypassing the transfer limits.

UM - Unusable Modifier

Criticality	Minor / Informative
Location	FRX-1.sol#L1439
Status	Unresolved

Description

The `_applyTax` function has the modifier `noReentrantTax`. However since the function calls `super._update` instead of `_update` the use of modifier is redundant.

Shell

```
function _applyTax(address from, uint256 amount)
    private
    noReentrantTax
    returns (uint256 net)
{
    uint256 burnAmt      = (amount * TAX_BURN)      /
BPS;
    uint256 recoveryAmt = (amount * TAX_RECOVERY) /
BPS;
    uint256 adminAmt     = (amount * TAX_ADMIN)     /
BPS;
    net = amount - burnAmt - recoveryAmt - adminAmt;

    if (burnAmt > 0) super._update(from,
address(0),      burnAmt);
    if (recoveryAmt > 0) super._update(from,
recoveryContract, recoveryAmt);
    if (adminAmt > 0) super._update(from,
feeReceiver,      adminAmt);
}
```

Recommendation

The team is advised to remove redundancies to enhance code readability and optimisation.

L04 - Conformance to Solidity Naming Conventions

Criticality	Minor / Informative
Location	FRX-1.sol#L1325,1332
Status	Unresolved

Description

The Solidity style guide is a set of guidelines for writing clean and consistent Solidity code. Adhering to a style guide can help improve the readability and maintainability of the Solidity code, making it easier for others to understand and work with.

The followings are a few key points from the Solidity style guide:

1. Use camelCase for function and variable names, with the first letter in lowercase (e.g., myVariable, updateCounter).
2. Use PascalCase for contract, struct, and enum names, with the first letter in uppercase (e.g., MyContract, UserStruct, ErrorEnum).
3. Use uppercase for constant variables and enums (e.g., MAX_VALUE, ERROR_CODE).
4. Use indentation to improve readability and structure.
5. Use spaces between operators and after commas.
6. Use comments to explain the purpose and behavior of the code.
7. Keep lines short (around 120 characters) to improve readability.

```
Shell
address _pair

uint _type
```

Recommendation

By following the Solidity naming convention guidelines, the codebase increased the readability, maintainability, and makes it easier to work with.

Find more information on the Solidity documentation

<https://docs.soliditylang.org/en/stable/style-guide.html#naming-conventions>.

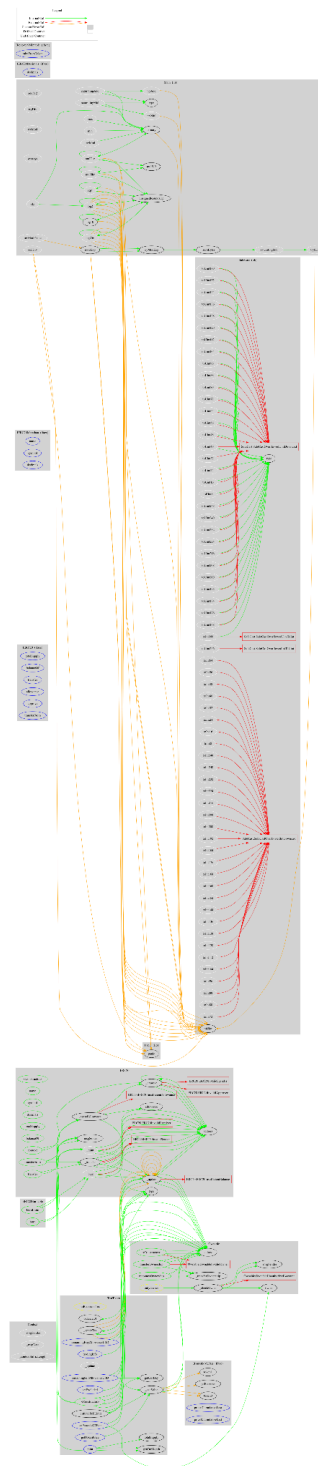
Functions Analysis

Contract	Type	Bases		
	Function Name	Visibility	Mutability	Modifiers
TestToken	Implementation	ERC20, ERC20Burnable, Ownable		
		Public	✓	ERC20 Ownable
	setPancakeV2Pair	External	✓	-
	setExcluded	External	✓	-
	mint	External	✓	-
	_update	Internal	✓	
	_enforceSellLimit	Private	✓	
	_applyTax	Private	✓	noReentrantTax
	_usdValue	Internal		
	getTokenPrice	External		-
	remainingSellAllowanceUSD	External		-
	remainingBuyAllowanceUSD	External		-
	getCurDay	Public		-
	getCurMonth	Public		-

Inheritance Graph



Flow Graph



Summary

The Fortune Protocol contract implements a token mechanism. This audit investigates security issues, business logic concerns and potential improvements.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security. Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



A **TAC Security** Company

The Cyberscope team

cyberscope.io